State Transition Testing

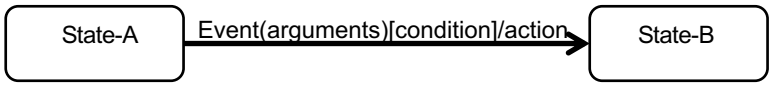
State Transition Testing

- Finite-state models are used to understand the various states the system has, including any which are initial and final
- Transitions, events, conditions and actions in each state are identified
- Graph or table are used to model the system and serve as an oracle – **state diagram**
- For each event and condition, verify action and next state

State transition diagram

- Has four basic parts:
  - the states that the software may occupy;
  - the transitions from one state to another;
  - the events that cause a transition;
  - the actions that result from a transition.

Transition notation



- Event:
  - triggers the transition
- Guard condition:
  - Transition only eligible to fire when guard evaluates to true
  - Guards of transition exiting one state are mutually exclusive
- Action:
  - executable atomic computation
  - May include operation calls, the creation or destruction of another objects, or the sending of a signal to an object.

Example for state modelling: Stack

Class `Stack`

State preserving operations

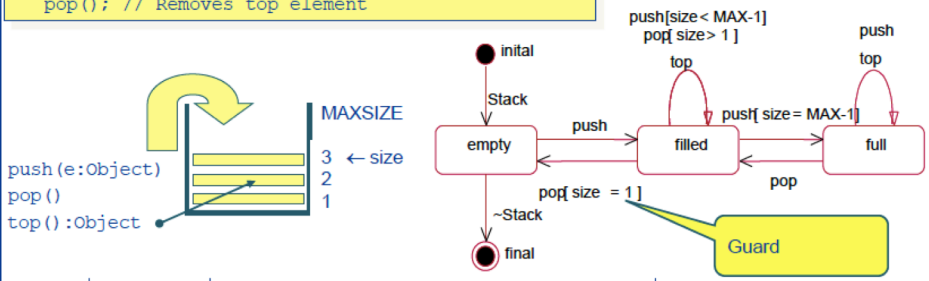
```
size():integer;  // Amount of stacked elements
MAX():integer;   // Maximum amount
top():Object;    // Arrow on top element
```

State transitioning operations

```
Stack(Max:integer);    // Constructor
~Stack();              // Destructor
push(element:Object);  // Stacks element
pop(); // Removes top element
```

Three states

```
empty:  size() = 0;
filled: 0 < size() < MAX();
full:   size() = MAX();
```



© Copyright 2011 to MSTB/GTB V 1.0

Objectives of the State Transition Testing

- Analysis, which proves that the test object complies with the state diagram (state-compliance testing)
- Additional testing under non-compliance applications (state-robustness testing)

© Copyright 2011 to MSTB/GTB V 1.0

Workflow of State Transition Testing

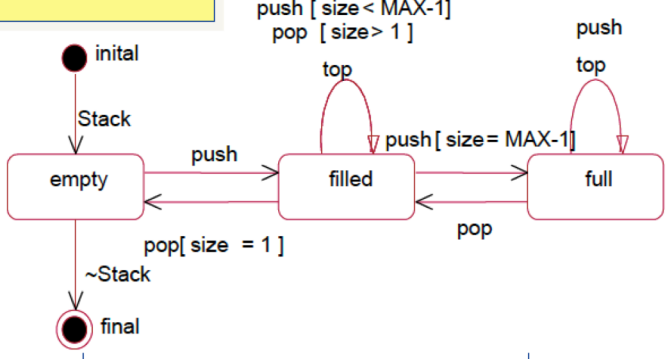
1. Focus on the state diagram
2. Test for completeness
3. Deriving the State Transition Tree for State Compliance Testing
4. Expand the State Transition Tree for the State Robustness Testing
5. Generate message sequences and add the message parameters
6. Execute the tests and measure achieved coverage

© Copyright 2011 to MSTB/GTB V 1.0

1. Focus on the State Diagram

Three States:
```
empty:  size() = 0;
filled: 0 < size() < MAX();
full:   size() = MAX();
```
Two „Pseudo-States“:
```
initial: Before Creation;
final:   After Deletion
```

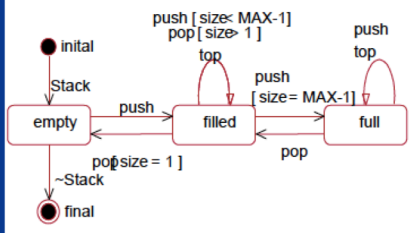
Eight State Transitions:
```
initial → empty; empty → final
empty → filled; filled → empty (cycle!)
filled → full; full → filled   (cycle!)
filled → filled; full → full   (cycle!)
```



© Copyright 2011 to MSTB/GTB V 1.0

2. Test for Completeness

- Testing the state diagram against its "completness"
  - If applicable, construct a state transition table
  - If applicable, also test the "guard" conditions regarding an event for "completeness" and consistency
  - Examine non-specified state/event-pairs



State / Event	initial	empty	filled	full
Stack()	empty	N/A	N/A	N/A
~Stack()	N/A	final	?	?
push()	N/A	filled	filled, full	full
pop()	N/A	?	empty, filled	filled
top()	N/A	?	filled	full

© Copyright 2011 to MSTB/GTB V 1.0

State Completeness Testing

Validation of state machines

Checklist for analyzing that the state machine is complete and consistent enough for model or implementation testing:

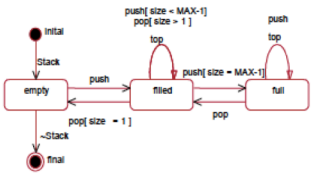
- one state is designated as the initial state with outgoing transitions
- at least one state is designated as a final state with only incoming transitions; if not, the conditions for termination shall be made explicit
- there are no equivalent states (states for which all possible outbound event sequences result in identical action sequences)
- every state is reachable from the initial state
- at least one final state is reachable from all the other states
- every defined event and action appears in at least one transition (or state)
- except for the initial and final states, every state has at least one incoming transition and at least one outgoing transition

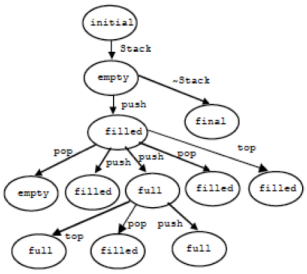
3. Structure of the State Transition Tree: State Compliance Testing

1. The start state will be the root of the tree.
2. The transition tree contains for every possible transition of the start state to the successor state in the state diagram from the root, via a branch to a node which represents the succesor state. At the branch, the event (operation) and, if applicable, the guard condition will be recorded.
3. The last step will be repeated for each leaf of the transition tree until one of the following two end conditions occurs:

   a. The state, which is represented by the leaf, is already contained in the tree at a „higher“ level.

   b. The state is an end state and therefore no further transitions are to be considered.



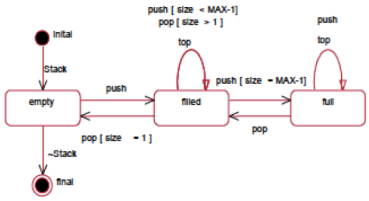


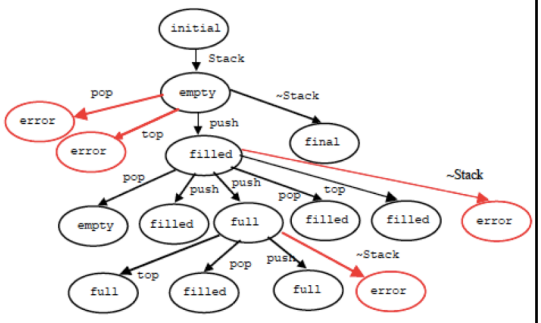
(guard conditions not shown here)

© Copyright 2011 to MSTB/GTB V 1.0

4. Expand the State Transition Tree: State Robustness Testing

- Testing the robustness under specific conditions of invalid use
- Expand the transition tree by a new „defect“-state for messages, for which is no valid transition is specified

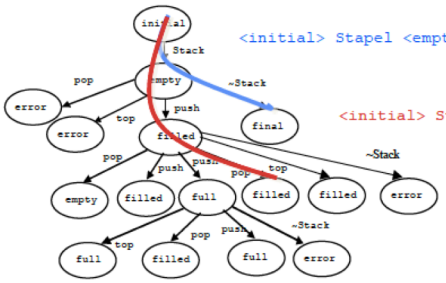




© Copyright 2011 to MSTB/GTB V 1.0

5. Generating the Test Cases(1)

- Paths from the root to the leaves in the expanded transition tree are considered as functional sequences
- Stimulating the test objects with the respective function calls will cover all states and state transitions in the state diagram
- Add parameters to the test cases!



<initial> Stapel <empty> ~Stack <final>

<initial> Stack <empty> push <filled> pop <filled>

© Copyright 2011 to MSTB/GTB V 1.0

5. Generating the Test Case (2)

- Paths from the root to the leaves in the expanded transition tree are considered as functional sequences
- Stimulating the test objects with the respective function calls will cover all states and state transitions in the state diagram
- Add parameters to the test cases!

State Compliance Testing

```
K1 = <initial> new Stack() <empty> ~Stack() <final>
K2 = <initial> new Stack() <empty> push() <filled> pop() <empty>
K3 = <initial> new Stack() <empty> push() <filled> push() <filled>
K4 = <initial> new Stack() <empty> push() <filled> pop() <filled>
...
K8 = <initial> new Stack() <empty> push() <filled> push() <full> push() <full>
```

State Robustness Testing:

```
R1 = <initial> new Stack() <empty> pop() <error>
R2 = <initial> new Stack() <empty> top() <error>
R3 = <initial> new Stack() <empty> push() <filled> ~Stackl() <defect>
R4 = <initial> new Stack() <empty> push() <filled> push() <full> ~Stack() <error>
```


© Copyright 2011 to MSTB/GTB V 1.0

State vs Transition coverage

- 100% state coverage -> only 1 test case
  - <initial>Stack<empty>push<filled>push<full>pop<filled>
  - Since by this test case, all states are tested.
- 100% transition coverage -> needs all 8 test cases

5. Generating the Test Case (3)

- Paths from the root to the leaves in the expanded transition tree are considered as functional sequences
- Stimulating the test objects with the respective function calls will cover all states and state transitions in the state diagram
- Add parameters to the test cases!´Observe the guard conditions!

State Compliance Testing
K1 = <initial> new Stack() <empty> ~Stack() <final>
K2 = <initial> new Stack() <empty> push() <filled> pop() <empty>
K3 = <initial> new Stack() <empty> push() <filled> push() <filled>
K4 = <initial> new Stack() <empty> push() <filled>
...
K8 = <initial> new Stack() <empty> push() <filled> push() <full> push() <full>

Guard conditions:
size() > 1 !!

☞push() <filled> pop() <filled>

State Robustness Testing:
R1 = <initial> new Stack() <empty> pop() <error>
R2 = <initial> new Stack() <empty> top() <error>
R3 = <initial> new Stack() <empty> push() <filled> ~Stackl() <defect>
R4 = <initial> new Stack() <empty> push() <filled> push() <full> ~Stack() <error>

© Copyright 2011 to MSTB/GTB V 1.0

State Transition Test Cases

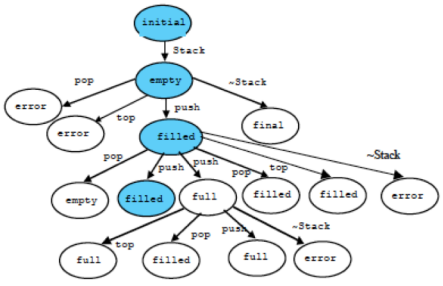
- A complete state transition test case contains the following information
  - Start state of the test object (component or system)
  - Input for the test object
  - Expected output, or expected response
  - Expected end state
- Furthermore, the following aspects are to be defined for each expected state transition in the test case
  - State before the transition
  - Triggering event, which results in the transition
  - Expected response, triggered by the transition
  - Next expected state

© Copyright 2011 to MSTB/GTB V 1.0

6. Execution of the Test

- Encapsulate test cases or message sequencies in a test script
- Execute by using a test driver
- Determine and record states and operations

```
K3' = //<initial>
        Stack OUT = new Stack(5)
      //<empty>
        OUT.push(new Object())
      //<filled>
        OUT.push(new Object())
      //<filled>
        if (OUT.size() != 2) then
         throw WrongStateException;
```



© Copyright 2011 to MSTB/GTB V 1.0

6. State Based Transition: Exit Criteria and Coverage Measuring

- Minimal criteria: Each state is captured at least once

State-coverage = (Amount of tested states / Total states) * 100 %

- Further criteria:
  - Each state transition was executed at least once

State Transition Coverage =
(Amount tested State Transitions / Total State Transition) * 100 %

  - All transitions were exercised which violate the specification
  - Each action (function) was executed at least once
- For highly critical applications
  - All state transitions and all »cycles« in the state diagram
  - All state transitions in each possible sequence with all possible states, even sucessive repetitions of the same state

© Copyright 2011 to MSTB/GTB V 1.0